

OBJETIVOS

- I/O: teclado.
- Tratamento de interrupção
- Criação de um *shell* muito simples.

TAREFA 1: IMPLEMENTAÇÃO DE E/S PARA O TECLADO (de 27/01/2025 a 30/01/2025)

Para a implementação de um *shell* simples, teremos que criar a primeira função de entrada de dados, similar a função *scanf()* ou *gets()* em C. A função a ser criada se chama *readStr()* pra a leitura de uma *string*. Além desta função, também será criada uma função acessório chamada *strEq()* a qual irá comparar duas *strings*.

A função *strEq()*, implementada na biblioteca *string.h*, é mostrada no Código 1. **Descreva o funcionamento desta função.**

```
uint8 strEq(string ch1, string ch2)
{
    uint8 result = 1;
    uint8 size = strlen(ch1);

    if (size != strlen(ch2))
        result = 0;
    else
    {
        uint8 i = 0;
        for(i; i <= size; i++)
            if (ch1[i] != ch2[i])
                result = 0;
    }

    return result;
}
```

Código 1: Código para a função *strEq()*.

Para a criação da função *readStr()*, esta deve ser inserida dentro do arquivo *kb.h*, no diretório *include* (Código 2). O objetivo desta função é traduzir os códigos de rastreamento de teclado (*scancode*) para código American Standard Code for Information Interchange (ASCII). Essa conversão é realizada no *switch()* que aparece no Código 2. Uma pequena amostra de como essa conversão deve ser realizada é apresentada dentro do *switch()*, para o *scancode* 1 (representando a tecla ESC). Para a tabela de *scancodes* completa consulte

<https://www.millisecond.com/support/docs/v5/html/language/scancodes.htm>. Você deve criar as entradas de conversão até a entrada 57 da tabela de *scancodes*. O *scancode* fornecido é para teclados americanos. Para outros idiomas, consulte o site da Microsoft. Além disso, perceba que a função de leitura é um “laço infinito”. Pense qual será a condição de parada para este laço. **O código resultante deve ser comentado no relatório, incluindo como foi definida a condição de parada desta função.**

Vale observar que, para ler dados do teclado, é necessário ler dados de duas portas diferentes: 0x64 e 0x60. O endereço 0x64 permite ler o *status* do teclado, para saber quando uma tecla é pressionada. O endereço 0x60 permite a leitura do “valor” da tecla pressionada. `inportb(0x64) & 0x1` indica se uma tecla foi pressionada. Esta operação *bitwise* resultará em 1 para qualquer *scancode* válido (não existe *scancode* com valor 0). `inportb(0x60)` fornece o valor do *scancode* propriamente dito, o qual será convertido para código ASCII.

```
#ifndef KB_H
#define KB_H
#include "screen.h"
#include "system.h"
#include "types.h"

string readStr()
{
    char buff[256];
    string buffstr = buff;
    uint8 i = 0;
    uint8 reading = 1;

    while(reading)
    {
        if(inportb(0x64) & 0x1)
        {
            switch(inportb(0x60))
            {
                /*case 1:
                    printch((char)27);
                    buffstr[i] =(char)27;
                    i++;
                    break;*/

                case 2:
                    ...

            }
        }
        buffstr[i] = 0;
        return buffstr;
    }
#endif
```

Código 2: Código parcial para a função `readStr()`.